Identifying the Visual Origin of Emojis

Josh SOUSSAN Edouard SOUIED

Computer Vision — Final Project

Part 1 — Abstract

The task is to predict which of seven vendors (Apple, Google, WhatsApp, Facebook, Samsung, Mozilla, Messenger) designed an emoji image, from 9,879 labelled training images, with accuracy as the metric. Our system fine-tunes ConvNeXt-Tiny networks on 4-channel RGBA input at two resolutions, adds nearest-neighbour features computed from frozen DINOv2 and CLIP embeddings, and combines everything with a LightGBM stacking model trained on out-of-fold predictions.

We deliver two versions. **Version A** uses the labelled training set only; it reaches **98.30 %** 5-fold out-of-fold (OOF) accuracy and **0.98663** on the public leaderboard, our best score. **Version B** additionally retrains the networks on confidently pseudo-labelled test images; it reaches 98.45–98.47 % OOF and 0.98582–0.98643 on the leaderboard. Pseudo-labelling therefore improved our internal estimate but not the leaderboard: the three final submissions differ by at most four images on the public half of the test set.

Two further findings shaped the work. First, the file itself is very informative: native image size and PNG encoding alone give 95.6 %. Because the brief asks for a prediction "based solely on visual characteristics", we report a pixels-only system next to the full one — it reaches 98.29 %, so metadata contributes almost nothing once a CNN is present. Second, a tempting transductive rule ("each emoji concept appears once per vendor") was tested and rejected because it lowers accuracy.

Part 2 — Summary / Table of contents

Part 1 — Abstract **1**

Part 2 — Summary / Table of contents **1**

Part 3 — Pre-processing (Data Augmentation) **2**
3.1 Data exploration 2
3.2 Input representation 4
3.3 Data augmentation 4
3.4 Validation protocol 5

Part 4 — Architecture of the model **5**
4.1 Overview 5
4.2 Fine-tuned CNNs 5
4.3 Frozen embeddings and kNN features 6
4.4 File and image statistics 6
4.5 Stacking 6
4.6 Transductive components 6

Part 5 — Metrics chosen **7**

Part 6 — Performance **7**
6.1 Ablation 7
6.2 Per-class results and error analysis 9
6.3 Leaderboard and the delivered versions 10
6.4 Limitations 11

Part 7 — AI Assessment **12**

Table 1: Pipeline at a glance.

Stage	What it does	Script
Index	PNG metadata, image statistics, pixel hashes, folds	`index.py`, `meta_features.py`
CNNs	ConvNeXt-Tiny on RGBA at 224 and 160 px, 5 folds	`dataset.py`, `train_cnn.py`
Embeddings	Frozen DINOv2 / CLIP → kNN features	`embed.py`, `concepts.py`
Stacking	LightGBM on CNN probabilities, kNN, statistics	`stack.py`
Pseudo-labels	Retrain CNNs on train + confident test images	`pseudo_label.py`
Submission	Duplicate rule, format checks, CSV	`make_submission.py`
Inference	Images → CSV from saved weights, no training	`predict.py`
Analysis	Ablation table and figures	`ablation.py`, `report_figures.py`

Part 3 — Pre-processing (Data Augmentation)

3.1 Data exploration

The training set is imbalanced: Apple, Google and Samsung have about 1,800–1,900 images each, while Messenger has 580 and Mozilla 397 (Figure 1).

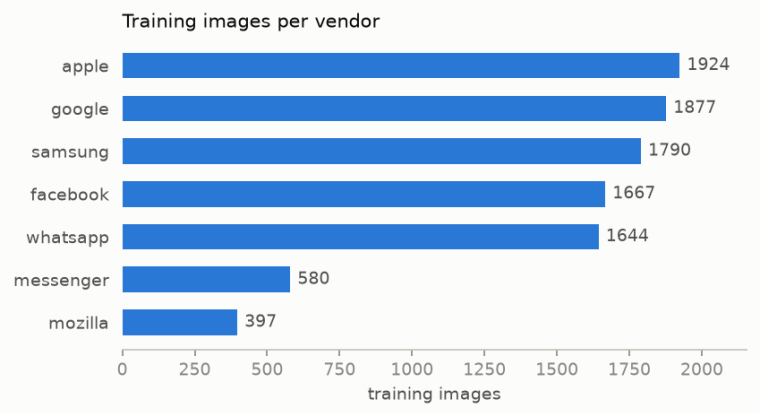


Figure 1: Training images per vendor.

Images are not delivered at a common size, and the native size is strongly tied to the vendor (Figure 2): 240×240 is almost exclusively Facebook, Messenger and Mozilla, 160×160 is WhatsApp, and 72×72 is Apple, Google and Samsung. The hard cases are the 7–9 % of WhatsApp, Facebook and Mozilla images that are stored at 72×72.

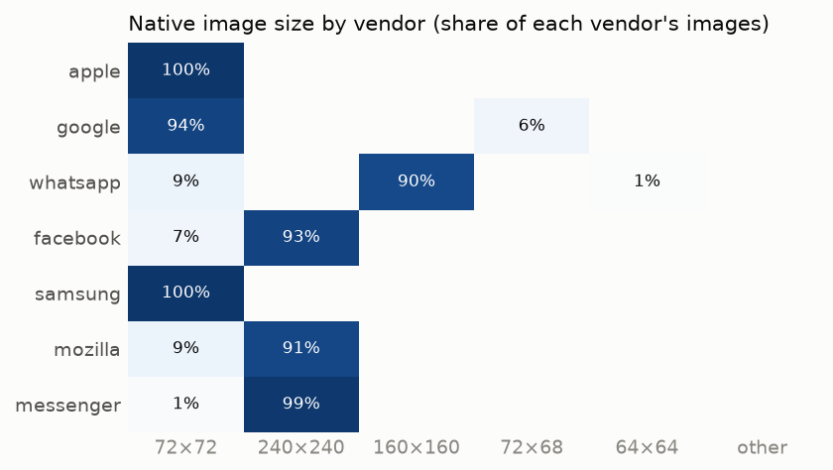


Figure 2: Native image size by vendor (share of each vendor's images).

Other observations that guided the design:

- **Encoding differs by vendor.** About 88 % of Samsung images use a colour palette (PNG mode `P`, at most 5 % for any other vendor); 46 % of Mozilla files carry a DPI chunk, which is almost absent elsewhere.
- **Exact duplicates exist.** 349 test images are pixel-identical to a training image, and duplicates inside the training set never disagree on the label.
- **Train and test look alike.** Size and mode distributions match, so a random stratified split is representative of the test set.



Figure 3: Random training samples per vendor.

3.2 Input representation

- **RGBA, four channels.** The alpha channel is kept as a network input instead of compositing on a background. Edge anti-aliasing, soft shadows and the exact silhouette are part of a vendor's rendering style.
- **Resizing.** Every image is resized once with bicubic interpolation to the network resolution (224 or 160 px) and cached as a `uint8` memory-mapped array.
- **Normalisation.** RGB channels use the ImageNet mean and standard deviation; alpha uses mean 0.5 and standard deviation 0.5.
- **Embeddings input.** For DINOv2 and CLIP only, images are composited on white, because those models are used to recognise *what* the emoji depicts, not how it is rendered.

3.3 Data augmentation

The label is the drawing style, so augmentations were chosen to leave style untouched (Table 2, Figure 4).

Table 2: Augmentation and regularisation settings.

Augmentation	Setting	Why
Random translation	$\pm 6\,\%$ of the image side	Emojis are not perfectly centred
Random scale	0.9–1.1	Vendors use different margins; avoids overfitting to exact size
Horizontal flip	$p = 0.5$	Style is preserved under mirroring
CutMix	$p = 0.2$ per batch	Regularisation; forces use of local texture cues
Label smoothing	0.1	Calibration, robustness to the few ambiguous labels
Colour / hue jitter	**not used**	The colour palette is itself a vendor signature

Augmentations run on the GPU on whole batches; padding introduced by the affine transform is transparent black. At test time each image is predicted twice (original and mirrored) and the probabilities are averaged.

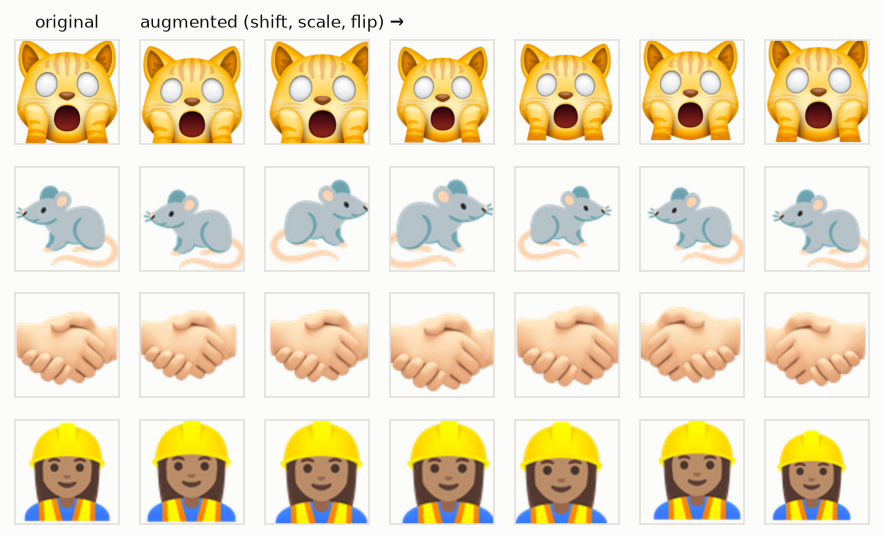


Figure 4: Augmentation examples: original (left) and six random augmentations.

Horizontal flipping was checked in isolation on one fold at 160 px: 97.37 % with flip against 97.27 % without, a difference of two images. It is kept but is not a significant lever.

3.4 Validation protocol

- **5-fold `StratifiedGroupKFold`**, stratified by vendor and grouped by pixel hash, so exact duplicates never appear on both sides of a split. A check in `index.py` asserts this.
- **The same folds are used by every model.** This makes the out-of-fold predictions of the CNNs valid inputs for the stacking model.
- **Class imbalance** is handled with moderate class weights (square root of inverse frequency) in the loss, and by monitoring per-class recall.

Part 4 — Architecture of the model

4.1 Overview

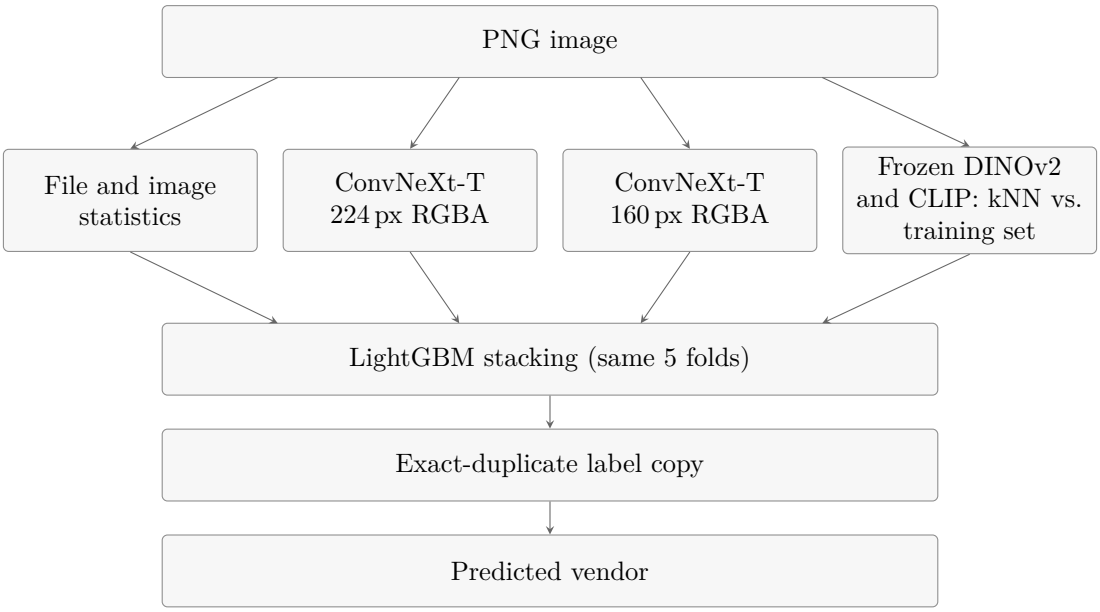


Figure 5: Version A. Version B adds ConvNeXt-T inputs trained on the training set plus pseudo-labelled test images.

4.2 Fine-tuned CNNs

- **Backbone:** ConvNeXt-Tiny, pre-trained on ImageNet-22k and fine-tuned on ImageNet-1k (`timm`), about 28 M parameters. The first convolution is adapted to four input channels and the classifier replaced by a 7-way head.
- **Two resolutions:** 224 px and 160 px, with different seeds, to obtain models whose errors are not identical.
- **Optimisation:** AdamW (learning rate 2×10^{-4}, weight decay 0.05), batch size 32, 30 epochs, one warm-up epoch then cosine decay, mixed precision, gradient clipping at 2.0, stochastic depth 0.1.
- **Weight averaging:** an exponential moving average of the weights (decay 0.999) is used for all predictions.

- **Hardware:** a single laptop RTX 3050 Ti (4 GB); about 65 s per epoch at 224 px and 30 s at 160 px.

ConvNeXt was chosen over heavier models because of the 4 GB memory limit and because, on this hardware, it trained several times faster than EfficientNetV2 or ResNet-50.

4.3 Frozen embeddings and kNN features

DINOv2 (ViT-S/14) and CLIP (ViT-B/32) are used without any training. For each image we retrieve its 20 nearest training images by cosine similarity and summarise them per vendor: maximum similarity, similarity-weighted vote, and neighbour counts in two similarity bands. The two bands matter because they carry opposite evidence:

- **Similarity ≥ 0.95:** the neighbour is from the *same* vendor 98 % of the time. These are variants of one drawing, typically skin-tone variants.
- **Similarity 0.8–0.9:** the neighbour is from a *different* vendor 65–80 % of the time — the same emoji drawn by someone else.

For training images, neighbours are only taken from the other folds, so the features are free of label leakage.

4.4 File and image statistics

About 140 features are computed without a neural network: native width and height, PNG mode, bit depth, chunk list and order, zlib header, palette size, alpha coverage, content bounding box, a 64-bin colour histogram, and gradient and edge statistics. They split into **PNG metadata** (properties of the file) and **pixel statistics** (properties of the decoded image), which lets us switch metadata off for the pixels-only system.

4.5 Stacking

A LightGBM multiclass model takes the log-probabilities of every CNN, the kNN features and the statistics, and is trained on out-of-fold rows with the same five folds. Test predictions are the average of the five fold models. The stacker learns interactions that a plain average cannot, for example trusting native size when the CNNs hesitate.

4.6 Transductive components

- **Exact duplicates.** A test image that is pixel-identical to a training image receives that image's label (349 images). In practice the models already predicted all of them correctly.
- **Pseudo-labelling (Version B only).** Test images that Version A predicts with at least 0.99 confidence are added to the training set with their predicted label: 9,046 of the 9,879 test images. On out-of-fold data, predictions above that threshold are correct 99.91 % of the time. The CNNs are retrained for 20 epochs on the enlarged set and added to the stack: the 224 px network alone in variant B1, both networks in variant B2.
- **Rejected: one vendor per emoji concept.** We grouped images by concept using the embeddings and solved a Hungarian assignment so that vendors within a group are all different. Simulated on out-of-fold data, this *lowers* accuracy by 0.7 to 1.0 points on Version A, because same-vendor variants end up in the same group. It is disabled; the kNN features keep the useful part of the idea in a soft form.

Part 5 — Metrics chosen

Metric	Why
Accuracy	The official challenge metric; used for every decision
Out-of-fold accuracy (5 folds)	Every training image is predicted by a model that never saw it, giving one estimate over all 9,879 images instead of a single small validation split
Per-class recall	Classes are imbalanced (397 to 1,924 images); accuracy alone could hide a weak small class
Macro-averaged recall	Single number that weights the seven vendors equally
Confusion matrix	Shows *which* vendors are confused, which drove the error analysis
CV–leaderboard gap	Checks that the validation protocol is trustworthy before relying on it

Accuracy differences are read with their noise in mind. One image is 0.01 points of OOF accuracy and about 0.02 points on the public leaderboard, which scores half of the test set (roughly 4,940 images). Differences of a few images are treated as ties.

Part 6 — Performance

6.1 Ablation

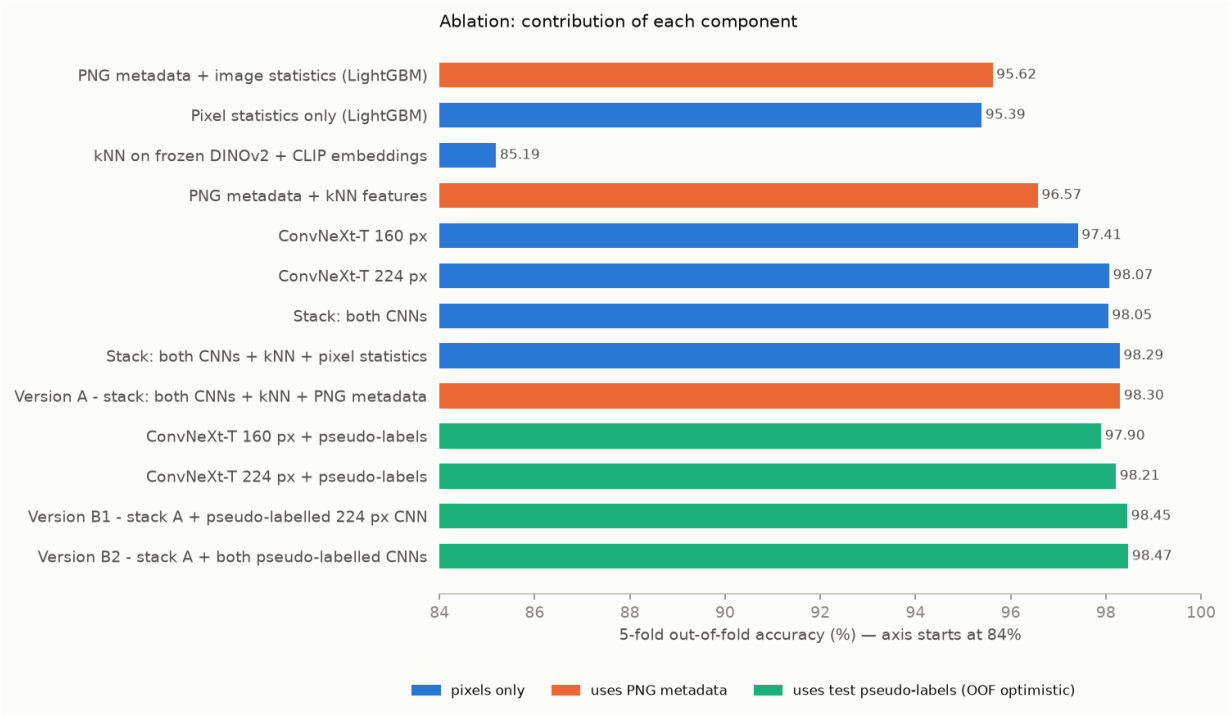


Figure 6: Contribution of each component (5-fold out-of-fold accuracy).

Table 3: Ablation. OOF accuracy of pseudo-labelled models is optimistic (Section 6.4).

Configuration	PNG metadata	OOF accuracy
PNG metadata + image statistics (LightGBM)	yes	95.62 %
Pixel statistics only (LightGBM)	no	95.39 %
kNN on frozen DINOv2 + CLIP embeddings	no	85.19 %
PNG metadata + kNN features	yes	96.57 %
ConvNeXt-T 160 px	no	97.41 %
ConvNeXt-T 224 px	no	98.07 %
Stack: both CNNs	no	98.05 %
Stack: both CNNs + kNN + pixel statistics	**no**	**98.29 %**
Version A: both CNNs + kNN + PNG metadata	yes	**98.30 %**
ConvNeXt-T 160 px + pseudo-labels	no	97.90 %
ConvNeXt-T 224 px + pseudo-labels	no	98.21 %
Version B1: A + pseudo-labelled 224 px CNN	yes	98.45 %
Version B2: A + both pseudo-labelled CNNs	yes	**98.47 %**

What the ablation shows:

- **The CNN is the main contributor.** A single ConvNeXt at 224 px reaches 98.07 %, 1.5 points above the best model without a CNN.
- **Resolution matters.** 224 px beats 160 px by 0.66 points.
- **kNN features add about 0.25 points** on top of the CNNs (98.05 % → 98.29 %).
- **PNG metadata adds nothing measurable once the CNNs are present:** 98.29 % without it, 98.30 % with it. The pixels-only system is therefore a fair answer to the brief.
- **The second CNN does not improve OOF accuracy** over the 224 px model with kNN features. It is kept because averaging two models is expected to be more stable on unseen data.
- **Pseudo-labelling raises OOF accuracy** of each CNN (+0.49 points at 160 px, +0.14 at 224 px) and of the stack (+0.17), but this gain did not transfer to the leaderboard (Section 6.3).

6.2 Per-class results and error analysis

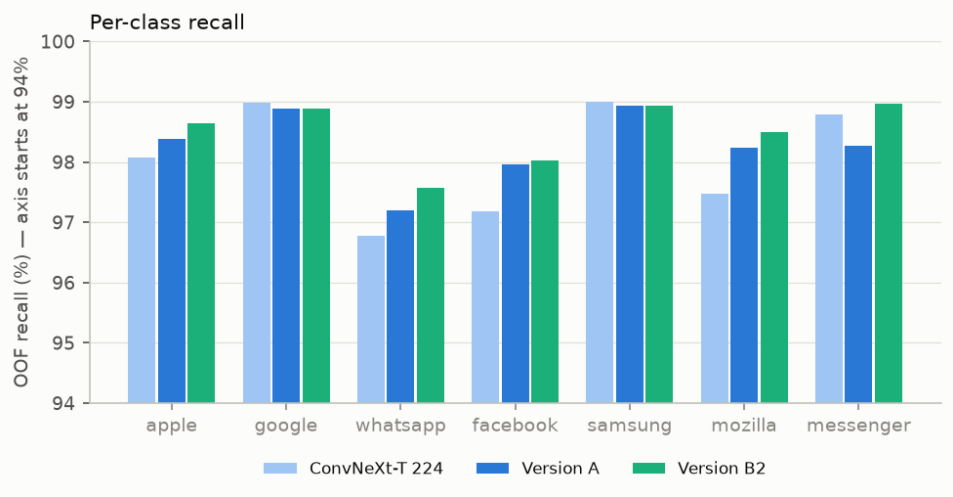


Figure 7: Out-of-fold recall per vendor.

	Errors (of 9,879)	Accuracy	Macro recall
Version A	168	98.30 %	98.27 %
Version B2	151	98.47 %	98.50 %

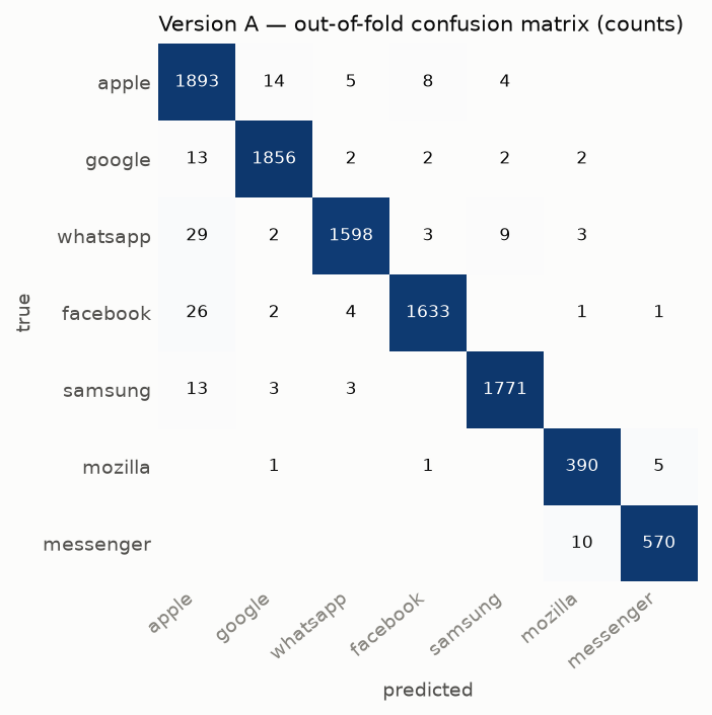


Figure 8: Version A, out-of-fold confusion matrix (counts).

- **WhatsApp and Facebook are the hardest classes** (97.2 % and 98.0 % recall in Version A), and their errors go mostly to Apple: 29 WhatsApp and 26 Facebook images are predicted as Apple.

- **Errors concentrate on 72×72 images:** 133 of the 168 errors, an error rate of 2.3 % against 0.85 % for all other sizes. At 72×72 the size cue is gone, and the WhatsApp and Facebook drawings stored at that size are visually very close to Apple's.
- **Mozilla and Messenger are confused almost only with each other** (15 of their 17 errors), consistent with both being flat-design sets delivered at 240×240.

The most confident errors of Version A are shown in Figure 9; most are 72×72 images whose true vendor is WhatsApp, Facebook or Samsung and which are predicted as Apple.



Figure 9: Most confident errors of Version A (true label, prediction with confidence, native size and PNG mode).

6.3 Leaderboard and the delivered versions

Table 4: Public leaderboard (half of the test set) against internal validation.

Submission	Model	Validation	Leaderboard
sub02	PNG metadata + DINOv2 kNN (LightGBM)	96.51 % (OOF)	0.96618
sub04	ConvNeXt-T 224, one fold only	98.28 % (fold 0)	0.98238
sub03	$0.7\times$ ConvNeXt fold 0 + $0.3\times$ metadata/kNN	98.53 % (fold 0)	0.98380
sub07	ConvNeXt-T 224 + kNN + pixel statistics, **no PNG metadata**	98.22 % (OOF)	0.98582
sub06	ConvNeXt-T 224 + DINOv2 kNN + metadata	98.32 % (OOF)	0.98582
sub05	ConvNeXt-T 224 + DINOv2/CLIP kNN + metadata	98.34 % (OOF)	0.98623
`final_A_no_pseudo.csv`	**Version A**	98.30 % (OOF)	**0.98663**
`final_B1_pseudo.csv`	**Version B1**	98.45 % (OOF)	0.98643
`final_B2_pseudo.csv`	**Version B2**	98.47 % (OOF)	0.98582

- **Version A is our best leaderboard score**, 0.28 points above the earlier single-fold blend, which is consistent with averaging five folds and learning the stack weights.
- **Pseudo-labelling did not help on the leaderboard.** Version B1 is one image below Version A and Version B2 four images below, on roughly 4,940 scored images. These gaps are within noise, so the three submissions are effectively tied, but the +0.17 point OOF gain of Version B is clearly not confirmed.
- **PNG metadata does not help on the leaderboard either.** The pixels-only stack (sub07) and the same stack with metadata (sub06) obtain exactly the same score, 0.98582, which confirms the ablation of Section 6.1 on unseen data.
- **A single CNN is almost as good.** The stack built on the 224 px ConvNeXt alone (sub05, 0.98623) is two images behind Version A: the second network adds very little.
- **Why the OOF gain was misleading.** The pseudo-labels are produced by models trained on all training folds, so a pseudo-labelled network indirectly benefits from the labels of its own validation fold. We flagged this bias before submitting and the leaderboard confirms it.
- **Validation otherwise tracked the leaderboard closely:** within 0.15 points on the first three submissions, and 0.36 points below it for Version A, the expected direction since test predictions average five fold models while OOF predictions come from one.

Versions A and B2 differ on 36 of the 9,879 test predictions (0.36 %).

Runnable models. `predict.py` reproduces predictions from saved weights with no training. Because the weights of the first 224 px network were not saved, it uses two stacks built only from networks whose weights are stored: `deploy_A` (ConvNeXt 160 px, kNN and metadata, no pseudo-labels, 98.05 % OOF) and `deploy_B` (ConvNeXt 160 px and both pseudo-labelled networks, 98.42 % OOF).

6.4 Limitations

- **OOF accuracy of pseudo-labelled models is optimistic**, as explained above. Their value should be judged on the leaderboard only.
- **Pseudo-labelling is transductive.** It uses the unlabelled test images, which is allowed here because the final evaluation is on that same test set, but Version B would have to be retrained to be applied to a different set of images. Version A has no such dependency.
- **Early stopping in the stacker** uses the validation fold, which biases the OOF accuracy of the stacks slightly upward. The same bias applies to every stack, so comparisons between them remain fair.
- **Training is not bit-reproducible.** Seeds are fixed, but cuDNN auto-tuning and mixed precision make GPU results vary by about ±0.1–0.2 points per fold between runs.
- **Remaining errors may include label noise.** Several high-confidence errors are 72×72 images that look like Apple drawings but are labelled WhatsApp or Facebook; we had no way to verify these labels.
- **The first 224 px network's weights were not saved**, only its predictions. All later networks (160 px and both pseudo-labelled ones) have their weights stored per fold.

Part 7 — AI Assessment

An AI coding assistant (Claude Code) was used throughout this project, strictly as an implementation tool. The plan was mine: I defined the overall approach and every step of the pipeline described in this report, decided what to try and in which order, and chose what to keep in the delivered versions. The assistant's role was to execute those steps in my place, that is, to write the code for each stage I specified, run it, and report the results back to me.

In short, the assistant replaced the typing, not the thinking: the design and the decisions are my own, and the code is the assistant's implementation of them.